

Influence-Aware Bayesian-Inspired Token Reweighting for Improved Code Generation

YUQI ZHU, Academy of Military Sciences, China

GE LI*, Peking University, China

HONG MEI, Peking University, China

ZHI JIN, Peking University, China

JIA LI, Wuhan University, China

QIBIN ZHENG, Academy of Military Sciences, China

JIEYUAN ZHANG*, Academy of Military Sciences, China

Large language models (LLMs) have achieved remarkable progress in code generation, yet the structural properties of programming languages introduce distinctive challenges. In particular, program correctness is disproportionately influenced by a subset of structurally critical tokens, such as API names, variable identifiers, and control-flow keywords, termed as influential tokens. Errors in predicting these tokens often propagate and accumulate through subsequent decoding steps, leading to substantial degradation in overall correctness. Addressing the heterogeneous difficulty of predicting such tokens is therefore crucial for improving the reliability of code generation. To address this challenge, we introduce Influence-Aware Bayesian Code Generation (I-BAYGEN), a framework that explicitly handles influential tokens. The framework consists of two components. First, it identifies influential tokens using a loss-based detection mechanism, and measures the influential degree of each token in three ways. Second, to handle influential tokens, we introduce auxiliary reasoning paths as additional evidence to refine the token distribution during code generation in a Bayesian-inspired manner. To capture structural dependencies, we incorporate influence scores as adaptive weights in the self-rewarding mechanism, encouraging greater optimization emphasis on structurally critical tokens. Using the influence-aware reweighting mechanism, the framework provides differentiated treatment to tokens based on their prediction difficulties, with influential tokens receiving enhanced attention through a reward weighting scheme and deeper reasoning processes. Comprehensive experiments on competition-level programming benchmarks demonstrate that I-BAYGEN achieves up to 47.2% relative improvement in correctness over state-of-the-art non-weighted approach. We further show that I-BAYGEN generalizes robustly across multiple programming languages and out-of-distribution scenarios, highlighting its potential for real-world code generation tasks. Moreover, qualitative analysis reveals that the framework produces reasoning paths that are more interpretable and logically coherent than non-weighted method, effectively addressing heterogeneous token difficulty in code generation.

CCS Concepts: • **Software and its engineering** → **Automatic programming**.

Additional Key Words and Phrases: Large Language Models, Code Generation, Reasoning-Augmented Generation

*Corresponding author.

*Corresponding author.

Authors' Contact Information: Yuqi Zhu, Academy of Military Sciences, Beijing, China, zhuyuqi@pku.org.cn; Ge Li, Peking University, Beijing, China, lige@pku.edu.cn; Hong Mei, Peking University, Beijing, China, meih@pku.edu.cn; Zhi Jin, Peking University, Beijing, China, zhijin@pku.edu.cn; Jia Li, Wuhan University, Wuhan, China, jia.li@whu.edu.cn; Qibin Zheng, Academy of Military Sciences, Beijing, China, zhengqb@aibd.ac.cn; Jieyuan Zhang, Academy of Military Sciences, Beijing, China, zhangjynudt@163.com.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE193

<https://doi.org/10.1145/3808200>

ACM Reference Format:

Yuqi Zhu, Ge Li, Hong Mei, Zhi Jin, Jia Li, Qibin Zheng, and Jieyuan Zhang. 2026. Influence-Aware Bayesian-Inspired Token Reweighting for Improved Code Generation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE193 (July 2026), 22 pages. <https://doi.org/10.1145/3808200>

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in natural language processing tasks [3, 17], leading to their widespread adoption for complex reasoning tasks such as mathematics [2] and code generation [28]. The application of LLMs to code generation has significantly enhanced developers' productivity [22], and further improving the code generation capabilities of LLMs holds considerable practical importance. However, applying LLMs to code generation presents unique challenges due to the distinct syntactic and semantic properties of programming languages, which differ substantially from natural languages.

Existing research [35] reveals a substantial difference between code generation and natural language generation. While prediction difficulty in natural language is relatively uniform across tokens, code generation exhibits significant variance in token prediction complexity throughout the generation process. We analyze token difficulty in code generation and identify tokens with high prediction difficulty as **influential tokens**. These influential tokens are characterized by higher prediction uncertainty, and errors in predicting them can cause the LLM's generation process to deviate from the oracle probability distribution. Errors in these tokens are particularly detrimental, as they can propagate to subsequent tokens and substantially degrade the overall quality and correctness of the generated code.

We find that such influential tokens often correspond to critical code elements such as API names, variable identifiers, and control-flow keywords, where prediction errors can cascade and compromise the semantic correctness of the generated code. The identification and proper handling of influential tokens represent a fundamental challenge in advancing code generation capabilities.

To address this challenge, we propose a novel two-stage framework: **Influence-Aware Bayesian Code Generation (I-BAYGEN)**. I-BAYGEN first detects influential tokens through a loss-based detection mechanism, then employs a Bayesian-inspired self-rewarding strategy that allocates reasoning computational resources proportionally to token difficulty.

Specifically, our detection mechanism computes per-token log loss to identify influential tokens, where loss magnitude serves as a principled proxy for epistemic uncertainty across diverse coding contexts. Meanwhile, we propose three methods for computing token-level influence scores: Linear Weighting(LW), Reciprocal Weighting(RW), and Superlinear Weighting(SW). Linear Weighting offers the advantage of computational simplicity, whereas Reciprocal Weighting provides increased sensitivity to tokens with low confidence. Superlinear Weighting further introduces a hyperparameter that controls the degree of emphasis placed on low-confidence tokens.

To handle influential tokens, we leverage the internal knowledge within the LLM and explicitly steer additional reasoning paths as auxiliary evidence to refine the code generation distribution, in a *Bayesian-inspired* manner [5]. From a Bayesian-inspired perspective, the model learns to optimize a posterior token distribution by integrating auxiliary reasoning signals with the posterior distribution. To enhance this posterior-like refinement, we integrate influence scores as adaptive weights within the self-rewarding framework [7], enabling the LLM to self-teach reasoning traces that pay greater attention to the identified influential tokens. By reweighting tokens according to their influence scores, the framework provides differentiated treatment based on token difficulty.

Through theoretical analysis, we find that the learning objective of Linear Weighting self-rewarding is equivalent to augmenting the original learning objective with a potential function that accelerates convergence, thereby satisfying the principles of potential reward shaping[23]. On the

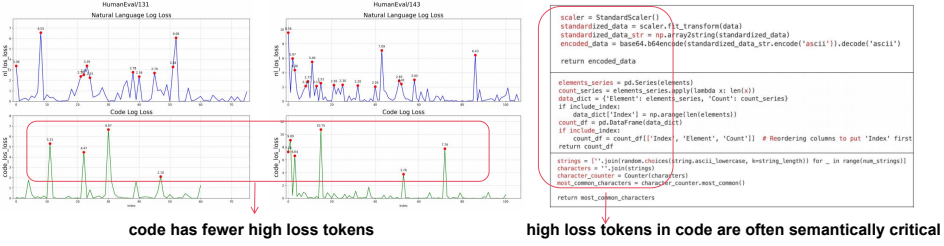


Fig. 1. The difficult tokens in code generation with LLMs are fewer than easy tokens.

other hand, the learning objective of Reciprocal Weighting self-rewarding corresponds to directly optimizing the Bayesian factor in the Bayesian inference process, which effectively maximizes the posterior probability.

Experimental evaluation across multiple code generation benchmarks, including the APPS[12] and LeetCode[11] benchmarks, demonstrates the consistent effectiveness of the I-BAYGEN. Specifically, we evaluate our method using reinforcement learning algorithms such as REINFORCE[1] and rejection sampling[20], and test it with LLMs including DeepSeek-Coder[11], QwenCoder[14], and CodeLlama[26]. Our method consistently improves both functional correctness and reasoning quality, achieving a 47.2% relative improvement in code generation accuracy compared to state-of-the-art non-weighted LATRO[8] method. Moreover, we conduct extensive hyperparameter analyses and assess our approach in multi-language setting using the XLCost dataset[34] and out-of-distribution (LiveCodeBench[15]) scenarios, successfully demonstrating its robust generalization capabilities. Furthermore, qualitative analysis demonstrates that I-BAYGEN generates more interpretable and logically coherent reasoning paths. These results highlight the importance of addressing the heterogeneous nature of token difficulty in code generation tasks, and suggest that our approach can effectively enhance model performance across a broad range of coding tasks.

The contributions of this work are threefold:

- We identify and analyze **influential tokens** in code generation tasks, highlighting their non-random distribution and their critical impact on code generation accuracy.
- We propose a novel **Influence-Aware Bayesian Code Generation (I-BAYGEN)** framework, which integrates loss-based influential token detection with a Bayesian-inspired self-rewarding mechanism to prioritize influential tokens and enhance overall generation accuracy.
- We demonstrate the effectiveness of our approach through extensive experiments on various challenging benchmarks and LLMs, and validate I-BAYGEN in multi-language and out-of-distribution scenarios, successfully demonstrating its robust generalization capabilities across diverse coding environments.

2 Motivating Example

We analyze the code generation process of large language models (LLMs) by comparing the loss dynamics during the generation of code versus natural language (NL). Our analysis is based on a subset of 50 randomly selected examples from both the HumanEval[9] and MBPP[4] datasets. For each ground-truth code solution, we manually write a corresponding natural language (NL) description following an imperative style. We then collect the token-wise log loss produced by the DeepSeekCoder-6.7B model when generating both the code and the corresponding NL description.

The results reveal that during code generation, the number of high-loss tokens is lower compared to NL generation. Moreover, the correct prediction of a high-loss token leads to a notably higher

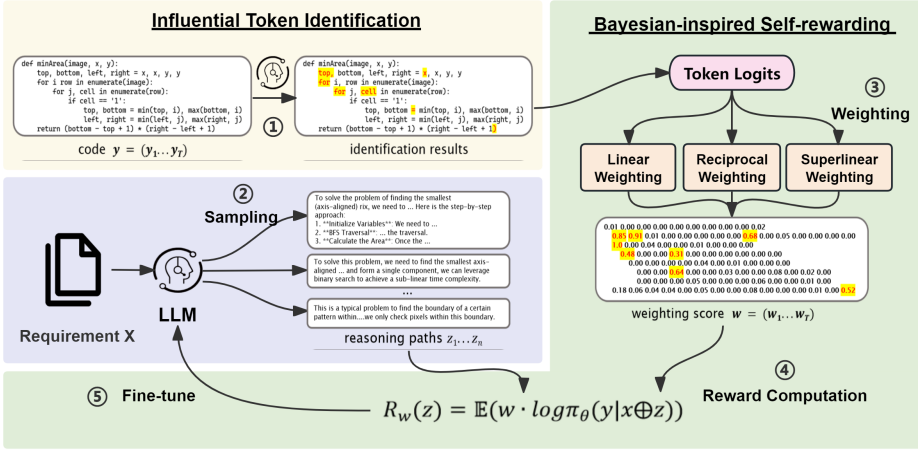


Fig. 2. The overview of Influence-Aware Bayesian Code Generation (I-BAYGEN).

average accuracy in the subsequent token predictions. We illustrate these patterns with several examples in Figure 1. The left panel of Figure 1 clearly demonstrates that code exhibits fewer high-loss tokens than NL. It also shows that when a high-loss token is correctly predicted, the losses of the following tokens are substantially reduced.

Another key finding is that high-loss tokens in code often correspond to semantically critical elements, such as variable names, API calls, and operators. These tokens are highlighted in red in the right panel of Figure 1, providing intuitive support for our observation. Based on these results, we define the concept of **influential tokens** in LLM code generation. These tokens are not only challenging for the model to generate correctly but also play a pivotal role in enhancing the correctness of subsequent tokens once accurately predicted. Typically, such influential tokens represent semantically critical components of the code.

The identification and proper handling of influential tokens open new avenues for improving code generation capabilities, suggesting that targeted interventions at these critical points could enhance overall generation performance.

3 Method

This section presents our proposed method, Influence-Aware Bayesian Code Generation (I-BAYGEN), a framework explicitly designed to address influential tokens. I-BAYGEN first identifies influential tokens through a loss-based detection mechanism and then adopts a Bayesian-guided self-rewarding approach that allocates computational resources proportionally to token importance. An overview of the framework is illustrated in Figure 2, including the token identification and Bayesian-guided self-rewarding modules. In the following subsections, we present a detailed explanation of our framework.

3.1 Influential Token Identification

In this section, we introduce the influential token identification method. The primary objective of this method is to identify the varying difficulty levels of different tokens relative to the LLMs'

coding capabilities. Based on the identification results, we compute influential values for each token, which reflect the degree of influence each token exerts during the LLM's generation process.

Given a LLM π_θ and training sample (x, y) , we estimate the prediction difficulty of each token y_t in the code snippet $y = (y_1, \dots, y_n)$ using the LLM's predictive confidence:

$$p_t = P_\theta(y_t \mid x, y_{<t}), \quad (1)$$

where x is the natural language requirement and π_θ denotes the LLMs' parameters. Tokens with lower predictive confidence correspond to higher uncertainty and are therefore considered more influential. This definition aligns with prior findings that errors on uncertain predictions tend to propagate more severely and compromise the correctness of subsequent generations.

We introduce three per-token weighting strategy $w_t = W(p_t)$ to capture token-level influence:

Linear weighting (LW). This strategy offers computational simplicity as its primary advantage, providing a straightforward linear relationship between token confidence and influence weight.

$$\textbf{Linear weighting (LW): } w_t = 1 - p_t \quad (2)$$

Reciprocal weighting (RW). This strategy addresses the limitation of Linear Weighting regarding invariant sensitivity to low-probability tokens. Employing reciprocal scaling provides enhanced sensitivity to tokens with very low confidence compared to the linear strategy.

$$\textbf{Reciprocal weighting (RW): } w_t = \frac{1}{p_t} \quad (3)$$

When implementing this method, we use $w_t = \frac{1}{p_t + \epsilon}$ to avoid division by zero, ϵ is a small constant.

Superlinear Weighting (SW). We further design this adjustable superlinear weighting method, where $\alpha > 1$ serves as a hyperparameter controlling the degree of emphasis on low-confidence tokens. This strategy provides tunable sensitivity, allowing fine-grained control over the influence attribution process.

$$\textbf{Superlinear Weighting (SW): } w_t = \left(\frac{1}{p_t} \right)^\alpha \quad (4)$$

Here, $\alpha > 1$ controls the degree of emphasis on low-confidence tokens. When implementing this method, we also use $p_t + \epsilon$ instead of p_t . These strategies assign greater emphasis to tokens that the LLMs find more difficult.

3.2 Bayesian-Guided Self-Rewarding

Since pretrained LLMs inherently possess reasoning capabilities that are not always explicitly activated or fully leveraged, we model code generation as a process in which a large language model (LLM) produces code $y = (y_1, \dots, y_T)$ from an instruction x , optionally accompanied by an internal reasoning trace c (e.g., a chain of thought).

In a Bayesian view, the standard autoregressive process defines a prior distribution:

$$P_\theta(y|x) = \prod_{t=1}^T P_\theta(y_t | y_{<t}, x) \quad (5)$$

while conditioning on a reasoning trace c induces a posterior distribution:

$$P_\theta(y|x, c) = \prod_{t=1}^T P_\theta(y_t | x, c, y_{<t}) \quad (6)$$

Intuitively, c plays the role of latent evidence or intermediate "hypotheses" that can update the model's beliefs about the correct code. Inspired by Bayesian principles, we view this process as a

form of distribution updating, where prior beliefs over hypotheses are refined in light of observed evidence. This perspective naturally accommodates uncertainty and enables the integration of both the LLM's implicit prior knowledge and its generated reasoning traces.

The learning objective of LLMs is to generate a reasoning trace c that maximizes the posterior probability of producing the correct code y . The optimization function can be formulated as:

$$\max_{c \sim \pi_\theta(\cdot|x)} P_\theta(y|x, c) = \max_{c \sim \pi_\theta(\cdot|x)} \prod_{t=1}^T P_\theta(y_t|x, c, y_{<t}) \quad (7)$$

Through our statistical analysis presented in Section 2, we identify a specific category of tokens that exert disproportionate influence on the code generation process. We term these influential tokens, which frequently correspond to critical code elements such as function names, variable declarations, and control structures. Prediction errors for these tokens are particularly detrimental, as they can propagate through the generation process and compromise the semantic correctness of the entire code sequence. Motivated by this insight, we incorporate token-specific influence weights into our optimization objective:

$$\max_{c \sim \pi_\theta(\cdot|x)} \sum_{t=1}^T w(y_t) \cdot P_\theta(y_t|x, c, y_{<t}) \quad (8)$$

where $w(y_t)$ represents the influence weight assigned to token y_t based on its identified influence on the overall generation quality.

Since the training signal is only observable after the full sequence c has been generated, this problem falls into the category of delayed reward. Moreover, this reward signal does not rely on external tools and can be derived solely from the LLM's own prediction probabilities. The self-rewarding[32] method provides a natural framework to solve this problem. The RL objective is:

$$\max_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathbb{E}_{c \sim P_\theta(c|x)} [\mathcal{R}(x, c, y)], \quad \mathcal{R}(x, c, y) = \sum_{t=1}^T w_t P_\theta(y_t|x, c, y_{<t}) \quad (9)$$

This weighted objective down-weights tokens that the prior already predicts well, and up-weights tokens that the reasoning trace must "explain." In practice, we use $\mathcal{R}(x, c, y)$ as the scalar reward function in the RL training of self-rewarding method (e.g., using REINFORCE or other RL methods). The objective encourages the model to produce reasoning traces c that assign higher probability to the correct code y , particularly for tokens where the prior is less informative.

In Section 3.1, we propose three influence weighting strategies, which represent complementary approaches for handling token-level uncertainty. Based on these strategies, we derive three corresponding self-rewarding objectives. We provide the theoretical analysis as follows.

Linear Weighting (LW) provides a simple method for adjusting token-level rewards according to model confidence. The sequence-level shaped reward under LW is:

$$\mathcal{R}_{LW}(x, c, y) = \sum_{t=1}^T (1 - p_t) P_\theta(y_t | x, c, y_{<t}), \quad (10)$$

where $p_t = P_\theta(y_t | x, c, y_{<t})$ denotes the model's predicted probability for token y_t .

The linear weighting self-rewarding can be interpreted as a special case of potential-based reward shaping. Potential-based Reward Shaping (PBRs) [23] provides a principled framework for redistributing delayed rewards to intermediate steps without changing the optimal policy. The general PBRs formula is:

$$\mathcal{R}'_t = \mathcal{R}_t + \gamma \Phi(s_{t+1}) - \Phi(s_t), \quad (11)$$

where $\Phi(s_t)$ is a potential function defined over states s_t , and here s_t represents the partially generated sequence $s_t = (x, y_1, \dots, y_{<t})$.

We define the token-level potential function as:

$$\Phi(s_t) = - \sum_{i=1}^t p_i P_\theta(y_i \mid x, y_{<i}), \quad (12)$$

and we set $\gamma = 1$. The resulting shaped reward for each token becomes:

$$\begin{aligned} \mathcal{R}'_t &= P_\theta(y_t \mid x, c, y_{<t}) + \Phi(s_t) - \Phi(s_{t-1}) = P_\theta(y_t \mid x, c, y_{<t}) - p_t P_\theta(y_t \mid x, c, y_{<t}) \\ &= (1 - p_t) P_\theta(y_t \mid x, c, y_{<t}), \end{aligned} \quad (13)$$

which matches the linear weighting token-level reward.

This derivation demonstrates that linear weighting can be interpreted as a special case of potential-based reward shaping, where the potential function accumulates token-level confidence-weighted probabilities. Intuitively, tokens with low model confidence receive higher shaped rewards, guiding the model to focus on learning these challenging tokens. PBRs [23] guarantees that the optimal policy under the shaped reward is identical to that under the original reward, while providing denser token-level training signals that can accelerate convergence.

Reciprocal Weighting (RW) emphasizes probabilistic rigor by linking directly to the Bayesian factor, thereby treating low-confidence tokens as stronger evidence in the belief-updating process. We define the Bayes factor for a given trace c and correct output y as:

$$BF(c; x, y) = \frac{P_\theta(y \mid x, c)}{P_\theta(y \mid x)} \quad (14)$$

The Bayes factor is a fundamental concept in Bayesian inference that quantifies the relative evidence provided by the data in favor of one hypothesis over another. It measures how much more likely the output y becomes conditioned on the reasoning trace c , compared to the baseline probability without this information. This weighting method allows us to rigorously assess the informational contribution of reasoning trace c using the principled machinery of Bayesian inference.

The reciprocal weighting self-rewarding objective is:

$$\mathcal{R}_{RW}(c) = \sum_{t=1}^T \frac{P_\theta(y_t \mid x, c, y_{<t})}{P_\theta(y_t \mid x, y_{<t})} \quad (15)$$

which equals directly maximize the bayesian factor $BF(c; x, y)$. Maximizing the bayesian factor encourages informative reasoning because it forces the model to shift its belief toward the correct solution in a focused way.

Equivalently, one can view each trace c as providing “evidence” for the hypothesis y . The Bayes factor is the strength of that evidence. By training the model to maximize $BF(c; x, y)$, we formally pushes the LLM to “explain” its code output step-by-step in a way that meaningfully increases its confidence in that output. This has a natural interpretability justification: the rewarded traces should logically connect x to y .

Superlinear Weighting, in turn, extends this principle by offering tunable control over emphasis, allowing the optimization to concentrate on the most challenging and influential tokens in a controllable manner.

The superlinear weighting self-rewarding objective is:

$$\mathcal{R}_{SW}(c) = \sum_{t=1}^T \frac{P_\theta(y_t \mid x, c, y_{<t})}{P_\theta(y_t \mid y_{<t}, x)^\alpha} \quad (16)$$

Collectively, these strategies not only reveal theoretical links between token weighting and Bayesian inference but also offer practical flexibility for tailoring the learning dynamics to diverse code generation contexts.

4 Study Design

To verify the effectiveness of I-BAYGEN, we conduct a large-scale study to answer four research questions. We next describe the experimental setup, including the datasets, baselines, LLMs, reinforcement learning methods, and evaluation metrics.

4.1 Research Questions

Our study mainly answers the following research questions (RQs).

- **RQ1: How does the I-BAYGEN method perform in code generation compared with state-of-the-art self-rewarding methods?** RQ1 aims to evaluate the effectiveness of the I-BAYGEN method in various competition-level code generation tasks across different LLMs.
- **RQ2: How does I-BAYGEN perform under different RL methods?** RQ2 aims to evaluate the effectiveness of I-BAYGEN when integrated with different RL methods.
- **RQ3: How sensitive is I-BAYGEN to different hyperparameter settings?** RQ3 examines the impact of key hyperparameters, including sampling size n , α , and ϵ , on the performance of I-BAYGEN.
- **RQ4: How robust is I-BAYGEN on out-of-distribution and multilingual datasets?** RQ4 investigates the generalization ability of I-BAYGEN when evaluated on datasets that differ from the training distribution or span multiple programming languages.
- **RQ5: What is the quality of the generated reasoning traces after learning with the I-BAYGEN method?** RQ5 evaluates the quality of the generated reasoning traces after learning with the I-BAYGEN method from both qualitatively and quantitatively compared to the baseline method.

4.2 Datasets

In the current experimental phase, this study focuses on competition-level code generation tasks with higher complexity. These challenging scenarios effectively examine LLMs' comprehensive performance in complex logical reasoning, algorithm design, and code generation. We use the following two datasets for our experiments:

- **LeetCode[11]** The LeetCode test set used in this work contains 180 high-quality problems from LeetCode contests between July 2023 and January 2024[11], covering core algorithmic topics such as data structures, dynamic programming, and graph theory. Each problem includes 100 curated test cases. The training set consists of 770 curated problems from the HuggingFace LeetCode dataset¹, preprocessed to ensure format consistency, difficulty balance, and strict separation from the test set to avoid data leakage.
- **APPS[12]** The APPS dataset comprises 10,000 programming problems ranging from introductory to competition-level difficulty. It is evenly split into 5,000 training and 5,000 test problems, with an average of 212 test cases per problem. For practical evaluation, we use the first 3,000 training problems and the first 1,000 test problems.
- **LiveCodeBench[15]** LiveCodeBench is a contamination-free benchmark for evaluating code generation models. To avoid pretraining data leakage, we use the latest LiveCodeBench V6, which includes problems released from April 2025 to now, all of which were published after

¹<https://huggingface.co/datasets/greengr0ng/leetcode/tree/main>

our collected training data. LiveCodeBench collects high-quality competitive programming problems from platforms such as LeetCode, AtCoder, and Codeforces.

- **XLCoST**[34] XLCoST is a multilingual code benchmark providing parallel code and natural language pairs across seven programming languages. In this work, we evaluate on the C++ and Java subsets of XLCoST, as both languages are widely used and representative in real-world software development.

4.3 Models

- **DeepSeekCoder**[11] is composed of a series of code LLMs (ranging from 1B to 33B). Each LLM is pre-trained on project-level code corpus by employing a window size of 16K and an extra fill-in-the-blank task, to support project-level code completion and infilling. In this paper, we use the DeepSeekCoder-6.7B-Instruct.
- **CodeLlama**[26] is a family of LLMs for code based on Llama 2 providing state-of-the-art performance among open LLMs, infilling capabilities, support for large input contexts, and zero-shot instruction following ability for programming tasks. In this paper, we use the CodeLlama-7B-Instruct.
- **QwenCoder**[14] is a new series of LLMs designed to achieve top-tier performance in coding tasks at various model sizes. Qwen2.5-Coder models are derived from the Qwen2.5 LLMs[30], inheriting their advanced architecture and tokenizer. These LLMs are trained on extensive datasets and further fine-tuned on carefully curated instruction datasets specifically designed for coding tasks. In this paper, we use the QwenCoder-Instruct model (0.5B, 7B, 14B).

We select models across different parameter scales (from 0.5B to 14B) to systematically evaluate the robustness and scalability of our method under varying model capacities, and to examine whether the proposed approach consistently improves performance across code LLMs of various sizes.

4.4 Baselines

We compare our method against the following baselines:

- **Self-planning** [16]: Generates code via the typical chain-of-thought method, which uses a '# Let's think step by step' and the end of the prompt to induce reasoning. While efficient, it may suffer from local optimality issues in complex tasks.
- **LATRO** [8]: Uses standard log-likelihood optimization over the final answer given the input and reasoning path, assigning equal importance to all tokens. This method has proven effective in math reasoning but lacks targeted optimization for difficult tokens.

4.5 Evaluation Metric

PassRate [9] measures the functional correctness of the generated code by executing test cases. For N problems, if the LLM successfully generate correct answers for C problems, PassRate is calculated as follows:

$$\text{PassRate} = \frac{C}{N} \quad (17)$$

In all experiments presented in this work, we use the greedy search decoding strategy. This choice eliminates the influence of sampling stochasticity on the final results. Additionally, greedy decoding imposes a higher level of reasoning difficulty for the LLM, as it is restricted to a single deterministic generation attempt without the benefit of multiple samples.

4.6 RL Methods

This section presents the reinforcement learning (RL) methods utilized in our experiments. All experiments were conducted on 4 NVIDIA A100 GPUs with 40 GB memory each.

We explore two widely-used reinforcement learning algorithms for fine-tuning code generation models: Rejection Sampling [20] and REINFORCE [1], both of which are adapted to integrate our token-level reward weighting mechanism.

Rejection Sampling is a sample-efficient policy optimization method where the model iteratively improves by fine-tuning on the best-performing completions under a predefined reward function. Given an input problem X , the model generates n candidate completions $C_1, C_2, \dots, C_n$, each evaluated by a reward function $R(X, C_i, Y)$. The candidate with the highest reward, $\hat{C} = \arg \max_i R(X, C_i, Y)$, is selected as the supervision signal for a supervised fine-tuning (SFT) step. This procedure is repeated over multiple rounds, and in our experiments, we fix the number of rounds to 2, as further iterations yield marginal performance improvements.

REINFORCE is a classic policy gradient method that directly optimizes the expected reward of the sampling distribution. For a sampled output c composed of tokens $y_{1:T}$, the model is updated using the gradient:

$$\nabla_{\theta} \mathcal{L}_{\text{RL}} = - \sum_{t=1}^T \mathcal{R} \cdot \nabla_{\theta} \log \pi_{\theta}(y_t \mid x, c, y_{<t}), \quad (18)$$

where $\mathcal{R}$ is a scalar reward signal.

4.7 RQ1: Effectiveness Across LLMs and Benchmarks

To assess the effectiveness of I-BAYGEN, we conduct comparative experiments on the LeetCode and APPS benchmarks with various LLMs. The results are summarized in Table 1.

To ensure efficient utilization of limited computational resources, we adopt the parameter-efficient Low-Rank Adaptation (LoRA) method [13]. Table 1 shows the results of the performance of I-BAYGEN and baselines on the LeetCode and APPS benchmarks. The superlinear and reciprocal weighting uses fixed hyperparameters: $\alpha = 2.0$, $\epsilon = 0.01$.

On LeetCode dataset, compared to the non-weighted baseline (LATRO), our I-BAYGEN achieves relative improvements on the Overall accuracy metric up to 19.2%, 47.2% and 6.8% on DeepSeekCoder, CodeLlama and QwenCoder, respectively, indicating consistent performance gains. On hard problems, weighted reward functions exhibit the most substantial improvements. Specifically, compared to LATRO, DeepSeekCoder gains 34.0% (LW), 100.0% (RW), and 67.6% (SW) relative improvements, while CodeLlama achieves 102.0% (LW), 51.1% (RW), and 102.0% (SW). QwenCoder achieves average improvements of 15.0% (LW), 22.3% (RW), and 15.0% (SW). **These results confirm that focusing optimization on low-confidence tokens substantially enhances performance on hard tasks requiring complex reasoning.**

For the APPS benchmark, compared to the non-weighted baseline (LATRO), I-BAYGEN improves Test Accuracy by up to 4.6%, 26.8%, and 4.3% on DeepSeekCoder, CodeLlama, and QwenCoder, respectively, demonstrating consistent gains across models. The Test Accuracy metric represents the average pass rate of the generated code across all test cases, while Strict Accuracy measures the proportion of cases where the generated code passes all test cases. APPS is widely recognized as a highly challenging benchmark, where prior RL-based methods (e.g., PPOCoder[27]) report low absolute performance (e.g., 1.32% and 3.27% pass@1), underscoring the task difficulty. Moreover, all results are obtained under greedy decoding (one solution per instance), without stochastic sampling (e.g., pass@100), making these improvements both non-trivial and practically meaningful. **These results demonstrate that I-BAYGEN's reward weighting mechanisms can consistently enhance code generation performance across different LLMs and challenging benchmarks.**

Table 1. Performance analysis of I-BAYGEN using different LLMs on the LeetCode and APPS benchmarks. All results are reported as percentages.

Model	Method	LeetCode				APPS	
		Overall	Easy	Medium	Hard	Test Acc.	Strict Acc.
DeepSeekCoder	Self-planning	16.1	37.8	9.9	6.8	20.7	4.9
	LATRO	17.2	42.2	9.8	6.8	21.9	5.1
	I-BAYGEN(LW)	20.5	44.4	14.3	9.1	22.1	5.6
	I-BAYGEN(RW)	20.5	42.2	13.2	13.6	22.9	5.9
	I-BAYGEN(SW)	19.4	46.6	9.8	11.4	22.4	5.5
CodeLlama	Self-planning	2.2	4.4	1.1	2.3	0.58	0.0
	LATRO	7.2	22.2	2.1	4.5	0.71	0.0
	I-BAYGEN(LW)	10.6	26.7	3.3	9.1	0.90	0.1
	I-BAYGEN(RW)	9.4	24.4	3.3	6.8	0.85	0.1
	I-BAYGEN(SW)	10.5	26.7	3.2	9.1	0.81	0.0
QwenCoder-7B	Self-planning	40.0	68.9	38.4	13.6	18.9	6.4
	LATRO	41.1	68.9	39.5	15.9	19.1	6.9
	I-BAYGEN(LW)	45.0	73.3	44.0	18.2	19.6	6.7
	I-BAYGEN(RW)	44.4	71.1	42.9	20.5	20.2	7.0
	I-BAYGEN(SW)	43.9	66.7	45.1	18.2	20.0	6.8
QwenCoder-14B	Self-planning	50.5	71.1	52.7	25.0	38.6	17.5
	LATRO	54.4	75.6	56.1	29.5	38.7	18.3
	I-BAYGEN(LW)	56.7	80.0	57.1	31.8	39.4	19.0
	I-BAYGEN(RW)	56.7	75.6	58.2	34.1	39.8	19.3
	I-BAYGEN(SW)	56.1	75.6	57.1	34.1	39.5	18.9

The effectiveness of different weighting strategies varies across LLMs and benchmarks.

On the LeetCode dataset, linear weighting (LW) generally achieves the best overall performance across models, for example, DeepSeekCoder improves from 17.2% (LATRO) to 20.5% overall accuracy, and QwenCoder-7B from 41.1% to 45.0%. In contrast, on the APPS dataset, reciprocal weighting (RW) performs better for most LLMs, particularly on hard problems, e.g., DeepSeekCoder’s hard problem accuracy rises from 9.1% (LW) to 13.6% (RW), a 49% relative increase, and QwenCoder-14B’s from 31.8% to 34.1% (7% relative increase).

We also measured the computational overhead of I-BAYGEN compared to the LATRO. On average per sample, the total time to compute rewards for the baseline method is 9.5227s, while I-BAYGEN requires 10.0211s, 9.8890s, and 9.6943s for LW, RW, and SW, respectively, corresponding to an overhead of approximately 1.8%-5.2%. This additional cost is negligible compared to the performance gains (up to 47.2%) achieved by I-BAYGEN.

4.8 RQ2: Effectiveness Across RL Methods

We use a widely used policy gradient method, REINFORCE[33], to test the general applicability of I-BAYGEN across different RL methods. Due to the computation constraints, we use the QwenCoder-Instruct (0.5B, 1.5B and 7B) and implement full parameter training. We fine-tune I-BAYGEN using a learning rate of 5×10^{-7} , batch size 16, for 2 epochs, with RLOO ($k = 16$), rollout batch size 16, gradient accumulation 12, and a KL coefficient of 0.05. The results are shown in Table 2.

Table 2. Performance analysis of REINFORCE using different reward weighting strategies on the LeetCode and APPS benchmarks. All results are reported as percentages.

Model Size	Method	LeetCode				APPS	
		Overall	Easy	Medium	Hard	Test Acc.	Strict Acc.
0.5B	Self-planning	5.6	15.6	3.3	0.0	8.5	0.5
	LATRO	6.1	22.2	1.1	0.0	8.8	0.5
	I-BAYGEN(LW)	7.8	26.7	2.2	0.0	9.7	0.6
	I-BAYGEN(RW)	6.7	22.2	1.1	2.3	9.3	0.6
	I-BAYGEN(SW)	7.2	24.4	1.1	2.3	9.5	0.6
1.5B	Self-planning	20.0	37.8	15.4	11.4	16.5	3.4
	LATRO	21.2	39.5	17.0	12.0	17.3	3.6
	I-BAYGEN(LW)	24.4	44.4	20.8	11.3	19.1	4.2
	I-BAYGEN(RW)	23.6	43.0	19.7	13.5	18.6	4.0
	I-BAYGEN(SW)	24.0	44.1	20.3	13.5	18.9	4.1
7B	Self-planning	40.0	68.9	38.4	13.6	18.9	6.4
	LATRO	40.5	71.1	39.5	11.4	19.2	6.9
	I-BAYGEN(LW)	42.7	73.3	42.9	11.4	19.5	6.6
	I-BAYGEN(RW)	43.3	71.1	44.0	13.6	20.8	7.1
	I-BAYGEN(SW)	43.9	71.1	42.9	18.1	21.2	7.3

Across different model scales (0.5B, 1.5B, and 7B), I-BAYGEN consistently improves over Self-planning and LATRO under the REINFORCE training framework. For the 0.5B model, I-BAYGEN increases LeetCode overall accuracy from 5.6% (Self-planning) to 7.8%, and improves APPS test accuracy from 8.5% to 9.7%. For the 1.5B model, the overall accuracy further rises from 20.0% to 24.4%, with consistent gains on both LeetCode and APPS. At the 7B scale, I-BAYGEN achieves the best overall performance of 43.9% on LeetCode and 21.2% test accuracy on APPS, surpassing both baselines. **These results demonstrate that our influence-aware reweighting strategy remains effective when replacing rejection sampling with a standard policy gradient method, confirming its general applicability across different RL algorithms and model sizes.**

4.9 RQ3: Hyperparameter Sensitivity Analysis

To further examine the robustness of I-BAYGEN, we conduct a sensitivity analysis on different hyperparameters, including number of sampled trajectories n , α and ϵ .

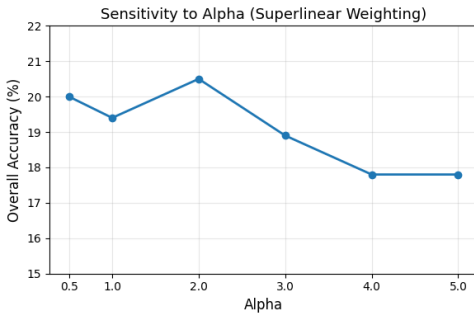
4.9.1 Number of Trajectory Sampling. Table 3 reports the performance of different reward functions when the sampling size increases from $n = 20$ to $n = 40$ using DeepSeekCoder-6.7B-Instruct with rejection sampling method on LeetCode benchmark.

The results reveal different sensitivities to the trajectory sampling number n across weighting strategies. Superlinear weighting (SW) benefits most from increased sampling, improving from 19.4% to 24.3% (+25.2% relative). Linear weighting (LW) shows more moderate gains (20.5% $\rightarrow$ 26.8%), while LATRO exhibits only limited improvement. In contrast, reciprocal weighting (RW) slightly declines as n increases, suggesting reduced robustness under larger sampling budgets.

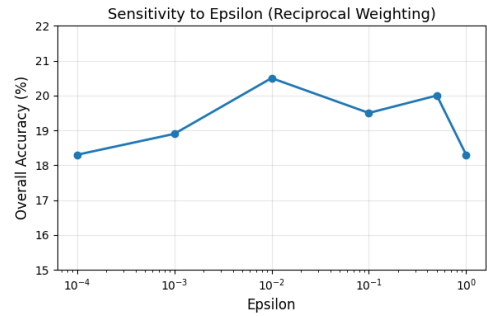
These findings provide two key insights. First, **the marginal utility of additional samples is method-dependent**: superlinear weighting benefits most from larger exploration spaces,

Table 3. Performance of different reward functions under different sample sizes on LeetCode benchmark. The reported metric is the OverAll pass rate. All results are reported as percentages.

Method	$n = 20$	$n = 40$	$\uparrow$ (%)
LATRO	17.2	19.1	+11.1%
I-BAYGEN (LW)	20.5	21.8	+6.3%
I-BAYGEN (RW)	20.5	19.8	-3.5%
I-BAYGEN (SW)	19.4	24.3	+25.2%



(a) Sensitivity to α (SW).



(b) Sensitivity to ϵ (RW).

Fig. 3. Sensitivity analysis of I-BAYGEN under different weighting hyperparameters on the LeetCode benchmark. Detailed breakdown across difficulty levels is provided in Table 4b, 4a.

Table 4. Hyperparameter sensitivity analysis of I-BAYGEN on the LeetCode benchmark.

(a) SW with different α values

α	Overall	Easy	Medium	Hard
0.5	20.0	46.7	12.1	9.1
1.0	19.4	46.7	9.8	11.4
2.0	20.5	42.2	13.2	13.6
3.0	18.9	46.7	12.1	4.5
4.0	17.8	42.2	9.9	9.1
5.0	17.8	40.0	11.0	9.1

(b) RW with different ϵ values.

ϵ	Overall	Easy	Medium	Hard
0.0001	18.3	37.8	13.2	9.1
0.001	18.9	40.0	13.2	9.1
0.01	20.5	42.2	13.2	13.6
0.1	19.5	42.2	13.2	9.1
0.5	20.0	44.4	12.1	11.4
1.0	18.3	42.2	11.0	9.1

whereas linear weighting achieves stable performance with smaller sampling budgets. Second, a **cost-benefit tradeoff must be considered in practice**: under ample computational resources, combining superlinear weighting with a larger sample size yields the strongest improvements, while in resource-constrained environments, the linear weighted strategy offers a more efficient and robust alternative.

4.9.2 α, ϵ . We have conducted a comprehensive evaluation of the key hyperparameters: α and ϵ in both weighting variants using DeepSeekCoder-7B under reject sampling on the LeetCode benchmark. The overall results are summarized in Figure 3. More detailed results are shown in Table 4a, 4b

Superlinear Weighting (α). We evaluated $\alpha \in \{0.5, 1.0, 2.0, 3.0, 4.0, 5.0\}$. The overall accuracy ranges from 17.8% to 20.5%. The best performance is achieved at $\alpha = 2.0$ (20.5%), while values in the range $\alpha \in [0.5, 3.0]$ consistently yield competitive results (18.9%–20.5%), with only modest fluctuations. Performance degradation becomes noticeable only when α is set excessively large (4.0 or 5.0), indicating that the method is stable across a reasonably wide interval.

Reciprocal Weighting (ϵ). We evaluated $\epsilon \in \{0.0001, 0.001, 0.01, 0.1, 0.5, 1.0\}$. The highest performance is obtained at $\epsilon = 0.01$ (20.5%). Importantly, the method maintains competitive performance for ϵ values in $[0.01, 0.1]$, while degradation is observed only when ϵ becomes too small or too large. Even in suboptimal settings, the performance drop remains moderate.

Overall, these results demonstrate that all weighting mechanisms exhibit strong robustness across a broad range of hyperparameter values. The method does not rely on delicate tuning and remains stable under practical parameter variations.

4.10 RQ4: Robustness Evaluation

4.10.1 Multilingual Settings. We conducted additional experiments on the **XLCoST**[34] multilingual code generation benchmark using **DeepSeekCoder-7B** and **QwenCoder-7B**. We evaluated two programming languages:

- **C++** (9.8k training instances / 909 test instances)
- **Java** (9.6k training instances / 911 test instances)

Since XLCoST does not provide executable test suites, we adopt **CodeBLEU** as the evaluation metric, including its four subcomponents: *ngram_match_score*, *weighted_ngram_match_score*, *syntax_match_score*, and *dataflow_match_score*. The results are shown in Table 5. The RL method we used is reject sampling.

On C++, the superlinear weighting achieves the best overall CodeBLEU score (0.288), improving over LATRO by **+4.3%**, **+3.9%** relatively for DeepSeekCoder and QwenCoder. Notably, improvements are consistent in *syntax* and *dataflow* scores, suggesting enhanced structural correctness rather than surface-level matching.

On Java, I-BAYGEN achieves a **+5.2%**, **+5.1%** relative improvement over LATRO in CodeBLEU for DeepSeekCoder and QwenCoder, with consistent gains across n-gram, syntax, and dataflow metrics. The increase in *syntax_match_score* (0.648, 0.430) and *dataflow_match_score* (0.528, 0.339) further indicates improved structural and semantic reasoning.

Overall, the results confirm that I-BAYGEN is not language-specific. Its benefits transfer consistently across languages with different syntactic and compilation properties. The stronger improvements observed in structural metrics further suggest that the method enhances deeper reasoning abilities instead of simply increasing token-level overlap.

4.10.2 Out-of-Distribution Evaluation. In this section, we test the performance of the I-BAYGEN method under an out-of-distribution (OOD) setting. We train QwenCoder-14B on the LeetCode training dataset and evaluate it on the LiveCodeBench [15] benchmark. The results are shown in Table 6. To avoid potential data leakage, we use the LiveCodeBench V6 test set (problems released after April 2025). Since all evaluated models and their training data were released prior to this time, this setup effectively mitigates the risk of data contamination.

Among the baseline methods, Self-planning and LATRO achieve modest PassRate scores of 14.8% and 15.4%, respectively. When incorporating the I-BAYGEN variants, we observe consistent improvements over both baselines: I-BAYGEN (LW) and I-BAYGEN (SW) reach 16.0% PassRate, while I-BAYGEN (RW) obtains the highest score at 16.6%. These results indicate that the I-BAYGEN approach enhances model robustness and yields better generalization to unseen problems compared with self-planning and LATRO.

Table 5. Code generation performance on XLCOST benchmark (C++ and Java).

Language	Model	Method	CodeBLEU	n-gram	w-gram	Syntax	Dataflow
C++	DeepSeekCoder	Self-planning	0.252	0.008	0.011	0.007	0.017
		LATRO	0.276	0.015	0.028	0.274	0.366
		I-BAYGEN (LW)	0.285	0.018	0.036	0.323	0.431
		I-BAYGEN (RW)	0.284	0.017	0.035	0.313	0.432
		I-BAYGEN (SW)	0.288	0.018	0.039	0.356	0.480
	QwenCoder	Self-planning	0.253	0.007	0.012	0.004	0.025
		LATRO	0.285	0.028	0.038	0.439	0.549
		I-BAYGEN (LW)	0.296	0.029	0.042	0.464	0.563
		I-BAYGEN (RW)	0.293	0.029	0.040	0.456	0.559
		I-BAYGEN (SW)	0.295	0.029	0.041	0.461	0.560
Java	DeepSeekCoder	Self-planning	0.235	0.147	0.178	0.205	0.410
		LATRO	0.536	0.468	0.547	0.621	0.508
		I-BAYGEN (LW)	0.558	0.496	0.576	0.640	0.524
		I-BAYGEN (RW)	0.560	0.495	0.577	0.640	0.528
		I-BAYGEN (SW)	0.564	0.500	0.582	0.648	0.528
	QwenCoder	Self-planning	0.211	0.156	0.199	0.264	0.218
		LATRO	0.353	0.310	0.357	0.412	0.331
		I-BAYGEN (LW)	0.365	0.324	0.373	0.422	0.336
		I-BAYGEN (RW)	0.363	0.321	0.370	0.418	0.339
		I-BAYGEN (SW)	0.371	0.331	0.379	0.430	0.336

Table 6. Out-of-Distribution evaluation results on the LiveCodeBench benchmark for models trained on LeetCode. All results are reported as percentages.

Method	PassRate
Self-planning	14.8
LATRO	15.4
I-BAYGEN (LW)	16.0
I-BAYGEN (RW)	16.6
I-BAYGEN (SW)	16.0

4.11 RQ5: Evaluation of Reasoning Traces

In this section, we evaluate the effectiveness of the reasoning traces generated by our method. The evaluation is conducted from both quantitative and qualitative perspectives.

4.11.1 Qualitative Analysis of Reasoning Trace. To evaluate the quality of the final generated reasoning traces, we adopt an LLM-as-a-Judge[18] evaluation approach. The LLM-as-a-Judge method uses a LLM to evaluate the quality of text outputs from another LLM, acting as an automated, scalable, and cost-effective alternative to human evaluators. Specifically, we compare the reasoning traces produced under the I-BAYGEN method with those from the unweighted self-rewarding method (LATRO) using GPT-4o². In this section, we use the linear weighting method as a representative example.

²<https://openai.com/index/hello-gpt-4o/>

Considering the structural feature code generation reasoning, we design a task-specific evaluation framework comprising four dimensions: Logical Coherence, Problem Analysis, Solution Planning, and Requirements Alignment. These definition of each dimensions are as follows.

- (1) **Logical Coherence:** Evaluates the structural and logical soundness of reasoning steps. (Max score: 10)
- (2) **Problem Analysis:** Assesses the understanding and decomposition of the programming problem, including constraints and edge cases. (Max score: 10)
- (3) **Solution Planning:** Measures the planning of the implementation strategy, such as algorithm choice and data structure design. (Max score: 10)
- (4) **Requirements Alignment:** Examines how well the reasoning matches task-specific requirements. (Max score: 10)

Instruction prompts are constructed to elicit structured evaluations from GPT-4o. To reduce potential bias, method names are anonymized (e.g., “Method A” vs. “Method B”). For consistency and reproducibility, GPT-4o outputs are constrained to a JSON schema:

```
{
  "method_a": {
    "logical_coherence": {
      "score": 8,
      "reasoning": "...",
    },
    "problem_analysis": {
      "score": 8,
      "reasoning": "...",
    },
    ...
    "total_score": 32
  },
  "method_b": {
    ...
  },
}
```

Table 7. Reasoning quality of generated outputs after training.

Dimension	I-BAYGEN	LATRO
Logical Coherence	8.23	6.15
Problem Analysis	8.35	5.69
Solution Planning	8.04	5.81
Requirements Alignment	8.54	6.15

We evaluate the generated reasoning traces of I-BAYGEN and LATRO on 180 held-out test samples from the LeetCode dataset. Table 7 summarizes the average scores, with Figure 4a and Figure 4b providing distributional and radar visualizations, respectively.

The results show that LLMs trained with the I-BAYGEN consistently outperform the unweighted self-rewarding method (LATRO) across all four evaluation dimensions, with scores consistently falling within the range of 8.2 to 8.5. This demonstrates that the LLMs guided by I-BAYGEN learns

Table 8. Reasoning quality comparison for different reward functions on 98 divergent samples.

Dimension	I-BAYGEN	LATRO
Logical Coherence	8.07	6.02
Problem Analysis	8.19	4.31
Solution Planning	8.11	6.00
Requirements Alignment	8.22	6.05

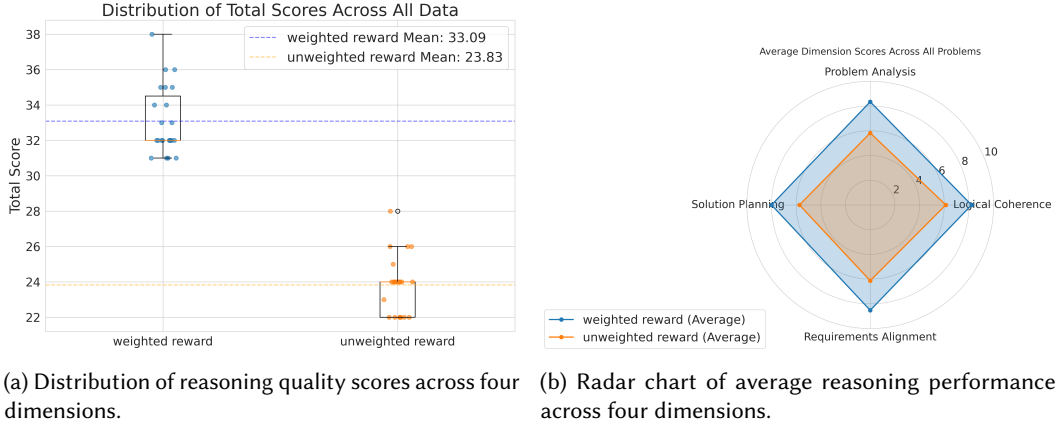


Fig. 4. Comparison of reasoning performance: distribution (a) and dimension-wise averages (b) under weighted vs. unweighted settings.

more coherent and problem-oriented reasoning traces. Notably, the largest improvement occurs in *Problem Analysis*, where our method achieves a relative increase of 48.6% over the baseline method, highlighting its ability to guide LLMs in decomposing complex programming tasks.

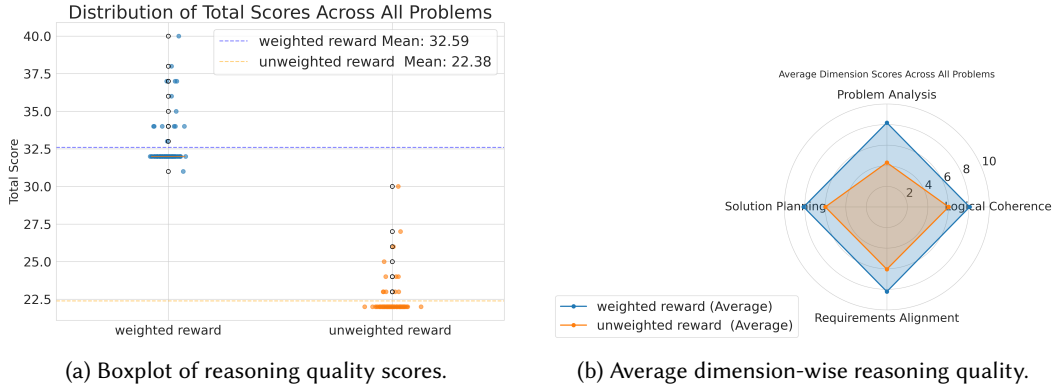


Fig. 5. Comparison of reasoning quality: overall scores (left) and dimension-wise averages (right).

4.11.2 Case Study. To gain deeper insights into the cases where the two methods diverge, we compare the reasoning traces selected by rejection sampling under the I-BAYGEN and LATRO reward schemes. In total, we identify 98 instances of differing selections. We then assess the quality of these divergent training samples. The results are summarized in Table 8, while Figure 5a and Figure 5b illustrate the overall score distributions and the per-dimension averages, respectively.

The weighted reward method achieves consistent improvements across all four dimensions, with the most pronounced gain in *Problem Analysis*. The 98 divergent training samples account for approximately 12.7% of the total training dataset (770 training samples). Although this proportion is relatively small, these high-quality reasoning traces are sufficient to contribute to the observed improvements in the final training outcomes.

LATRO

To solve this problem efficiently, we can use a binary search approach. This will minimize the number of calls to the API by repeatedly dividing the search interval in half.

I-BAYGEN

To solve the problem of finding the first bad version with the minimum number of calls to the `isBadVersion` API, we can use a binary search approach. Binary search is efficient for this problem because it reduces the search space by half with each iteration. Here's a step-by-step explanation of the approach:

1. Initialize two pointers, `left` and `right`, to represent the range of versions we are searching. Initially, `left` is set to 1 and `right` is set to `n`.
2. While `left` is less than or equal to `right`:
 - Compute `mid = left + (right - left) // 2` to avoid overflow.
 - Call `isBadVersion(mid)`. If `True`, update `right` to `mid`; otherwise update `left` to `mid + 1`.
3. When the loop exits, `left` will point to the first bad version.

Fig. 6. Cases of the generated reasoning traces by LATRO and I-BAYGEN.

LATRO

To solve this problem, we need to find the maximum possible sum of elements in a balanced subsequence of the given array 'nums'. A subsequence is considered balanced if the difference between any two consecutive elements in the subsequence is greater than or equal to the difference between their indices.

I-BAYGEN

To solve this problem, we need to determine if it's possible to sort the array using a specific set of operations. The operations allowed are swapping any two adjacent elements if they have the same number of set bits.

1. **Count the number of set bits** for each number.
2. **Sort by set bit count**.
3. **Check sortedness** of the resulting array.

Fig. 7. Cases of the generated reasoning traces by LATRO and I-BAYGEN.

To illustrate the qualitative differences, we present several reasoning traces generated by LATRO and I-BAYGEN. Figure 6 highlights a case where I-BAYGEN produces a more detailed step-by-step reasoning process, while Figure 7 shows improved alignment between problem requirements and solution strategy.

5 Discussion

Our work leverages a self-rewarding framework, which offers several practical advantages. First, it does not require large amounts of labeled reasoning trace training data, making it feasible

even in data-scarce scenarios. This reduces the cost and effort associated with data collection and annotation, which are often major bottlenecks in training code generation models. Second, it avoids the need to generate test cases during training, thereby eliminating potential errors that could be introduced by incorrect or incomplete test cases. This also enhances the stability and reproducibility of the training process, as the model does not rely on potentially noisy external supervision signals.

The effectiveness of our method stems not from teaching the model new knowledge, but from aligning it to better leverage existing knowledge. By identifying and emphasizing influential tokens, our approach guides the model to focus its reasoning on structurally critical components of the code. Moreover, the self-rewarding mechanism is inherently flexible and model-agnostic. It can be integrated with various backbone LLMs without requiring architectural changes. This flexibility allows practitioners to adapt the framework to different model scales and deployment scenarios. Finally, by improving the handling of influential tokens, our method not only boosts quantitative performance metrics but also improves the interpretability of the model's reasoning process, offering insights into which code components are most critical and how the model prioritizes them during generation.

6 Related Work

6.1 Code Generation with LLM

The rapid development of LLMs, such as GPT-3 [6] and GPT-4 [24], has significantly advanced the field of code generation. In recent years, to more precisely meet programming needs, researchers have developed Code LLMs specifically optimized for programming scenarios. These Code LLMs have demonstrated exceptional capabilities in various specific contexts, such as multi-language programming, code completion, debugging and fixing, and code refactoring [9]. Through training on large-scale codebases, Code LLMs can understand complex programming logic, syntactic structures, and developer intents, thereby significantly lowering the barrier to development and improving development efficiency [10]. Currently, representative Code LLMs include CodeLlama [26], DeepseekCoder [11], and QwenCoder [30], which have been widely adopted in both open-source communities and the industry. To advance the performance of LLMs in code generation tasks, software engineering research has introduced specialized enhancement techniques. These techniques focus on three key methodologies: First, prompting techniques to integrate software engineering expertise, such as domain knowledge and design patterns, to guide LLMs toward more complex solutions [16]. Second, targeted fine-tuning optimizes LLMs using domain-specific datasets, sharpening their ability to generate contextually relevant code [29]. Finally, program-specific decoding leverages test cases, runtime feedback, and probabilistic sampling strategies to refine outputs, enhancing the accuracy and soundness of the model-generated code [35].

6.2 Reasoning via Self-Rewarding

Recent work has explored self-improvement strategies for LLM reasoning. Zelikman introduced the Self-Taught Reasoner (STaR)[32], which iteratively generates chain-of-thought rationales and fine-tunes the model on those that produce correct answers; its successor Quiet-STaR[31] generalizes this idea to infer latent reasoning steps in arbitrary text. Similarly, Trung proposes Reinforced Fine-Tuning (ReFT)[21], which leverages reinforcement learning to reward exploration of diverse solution paths. These approaches adjust model parameters during training, in contrast to inference-time techniques such as self-planning[16] and SCoT[19]. Traditional alignment approaches like RLHF[25] rely on external reward models or human feedback, but designing reliable reward functions for complex reasoning is challenging. Chen [7] thus introduces LaTRO, a variational self-rewarding framework that treats reasoning paths as latent variables and uses the model's own

answer likelihoods as intrinsic rewards. This enables the model to improve its rationale generation and answer evaluation concurrently without external supervision, aligning with latent-variable modeling efforts while uniquely exploiting the model’s internal confidence as the learning signal.

7 Threats to Validity

There are two main threats to the validity of our work.

Threats to external validity mainly concern the quality and diversity of the datasets. For our study, we select two challenging benchmarks for LLMs: LeetCode and APPS. These datasets include programming problems that require a broad range of capabilities, such as handling variability in natural language inputs, understanding newly defined contexts, demonstrating commonsense reasoning, following complex instructions, applying mathematical and algorithmic knowledge, and exhibiting familiarity with coding principles. Such diversity ensures that our proposed method is generalizable across different types of programming tasks. To validate the effectiveness of our approach, we consider models from multiple LLM families, including DeepSeekCoder, CodeLlama, and QwenCoder. We adopt PassRate as the evaluation metric, an execution-based measure that checks the correctness of generated programs against test cases, ensuring the reliability of our results.

Threats to internal validity relate to the influence of hyperparameter settings. While we select configurations that produce satisfactory results, different settings may lead to performance variations. In our experiments, we maintain consistent hyperparameters across all training procedures, and we also conduct a detailed analysis of the most important hyperparameters. This careful examination helps to reduce potential biases, mitigate the influence of randomness, and ensures a fair comparison between methods.

8 Conclusion

In this work, we addressed the challenge of heterogeneous token difficulty in LLM-based code generation by introducing Influence-Aware Bayesian Code Generation (I-BAYGEN). Our framework explicitly identifies and models influential tokens that disproportionately affect program correctness. By integrating loss-based detection, influence scoring, and Bayesian-inspired reward reweighting, I-BAYGEN enables LLMs to allocate greater attention and reasoning capacity to influential tokens. Experimental results on competition-level programming benchmarks demonstrated substantial gains, with up to 47.2% relative improvement in accuracy over existing non-weighted self-rewarding methods. Furthermore, we conducted a detailed hyperparameter analysis and demonstrated the generalizability of our method across different LLMs and benchmark settings. Beyond quantitative improvements, our qualitative analyses confirmed that I-BAYGEN generates more interpretable and logically coherent reasoning paths, offering enhanced transparency in code generation.

The principles of influence-aware learning open promising directions for future research. The proposed framework can be extended beyond code generation to other critical tasks in software engineering. For example, it could enhance automated program repair, bug detection, and code summarization by enabling models to focus on structurally influential tokens that determine correctness and comprehensibility.

9 Data Availability

Our code is available at <https://github.com/yyyyqi/I-BAYGEN>.

Acknowledgments

This work is supported by the National Natural Science Foundation of China under Grant No.92582203, 62332012.

References

- [1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 12248–12267. doi:10.18653/V1/2024.ACL-LONG.662
- [2] Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. 2024. Large Language Models for Mathematical Reasoning: Progresses and Challenges. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*. 225–237.
- [3] Yadagiri Annapaka and Partha Pakray. 2025. Large language models: a survey of their development, capabilities, and applications. *Knowledge and Information Systems* 67, 3 (2025), 2967–3022.
- [4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [5] George EP Box and George C Tiao. 2011. *Bayesian inference in statistical analysis*. John Wiley & Sons.
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.
- [7] Haolin Chen, Yihao Feng, Zuxin Liu, Weiran Yao, Akshara Prabhakar, Shelby Heinecke, Ricky Ho, Phil Mui, Silvio Savarese, Caiming Xiong, et al. 2024. Language models are hidden reasoners: Unlocking latent reasoning capabilities via self-rewarding. *arXiv preprint arXiv:2411.04282* (2024).
- [8] Haolin Chen, Yihao Feng, Zuxin Liu, Weiran Yao, Akshara Prabhakar, Shelby Heinecke, Ricky Ho, Phil Mui, Silvio Savarese, Caiming Xiong, and Huan Wang. 2024. Language Models are Hidden Reasoners: Unlocking Latent Reasoning Capabilities via Self-Rewarding. *CoRR abs/2411.04282* (2024). doi:10.48550/ARXIV.2411.04282 arXiv: 2411.04282.
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR abs/2107.03374* (2021). <https://arxiv.org/abs/2107.03374> arXiv: 2107.03374.
- [10] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *ICSE-FoSE*. IEEE, 31–53.
- [11] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR abs/2401.14196* (2024). arXiv:2401.14196 doi:10.48550/ARXIV.2401.14196
- [12] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c24cd76e1ce41366a4bbe8a49b02a028-Abstract-round2.html>
- [13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. <https://openreview.net/forum?id=nZeVKeeFYf9>
- [14] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. *CoRR abs/2409.12186* (2024). arXiv:2409.12186 doi:10.48550/ARXIV.2409.12186
- [15] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=chfJYC3iL>

- [16] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-Planning Code Generation with Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 33, 7 (2024), 182:1–182:30. doi:10.1145/3672456
- [17] Pranjal Kumar. 2024. Large language models (LLMs): survey, technical frameworks, and future challenges. *Artificial Intelligence Review* 57, 10 (2024), 260.
- [18] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. Llms-as-judges: a comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579* (2024).
- [19] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured Chain-of-Thought Prompting for Code Generation. *ACM Trans. Softw. Eng. Methodol.* 34, 2 (2025). doi:10.1145/3690635
- [20] Tianqi Liu, Yao Zhao, Rishabh Joshi, Misha Khalman, Mohammad Saleh, Peter J. Liu, and Jialu Liu. 2024. Statistical Rejection Sampling Improves Preference Optimization. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=xbjSwwrQOe>
- [21] Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. 2024. Reft: Reasoning with reinforced fine-tuning. *arXiv preprint arXiv:2401.08967* (2024).
- [22] Amr Mohamed, Maram Assi, and Mariam Guizani. 2025. The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Literature Review. *arXiv preprint arXiv:2507.03156* (2025).
- [23] Andrew Y Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, Vol. 99. Citeseer, 278–287.
- [24] OpenAI. 2023. GPT-4 Technical Report. CoRR abs/2303.08774 (2023). arXiv:2303.08774 doi:10.48550/ARXIV.2303.08774
- [25] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [26] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. CoRR abs/2308.12950 (2023). arXiv:2308.12950 doi:10.48550/ARXIV.2308.12950
- [27] Parshin Shojaee, Aneesh Jain, Sindhu Tipirneni, and Chandan K Reddy. [n. d.]. Execution-based Code Generation using Deep Reinforcement Learning. *Transactions on Machine Learning Research* ([n. d.]).
- [28] Jianxun Wang and Yixiang Chen. 2023. A review on code generation with llms: Application and evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 284–289.
- [29] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. 2025. Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models. *ACM Trans. Softw. Eng. Methodol.* (2025). doi:10.1145/3714461
- [30] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. CoRR abs/2412.15115 (2024). arXiv:2412.15115 doi:10.48550/ARXIV.2412.15115
- [31] Eric Zelikman, Georges Raif Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah Goodman. [n. d.]. Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking. In *First Conference on Language Modeling*.
- [32] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. 2022. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems* 35 (2022), 15476–15488.
- [33] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. 2022. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems* 35 (2022), 15476–15488.
- [34] Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. 2022. XLCoS: A Benchmark Dataset for Cross-lingual Code Intelligence. CoRR abs/2206.08474 (2022). arXiv:2206.08474 doi:10.48550/ARXIV.2206.08474
- [35] Yuqi Zhu, Jia Li, Ge Li, Yunfei Zhao, Zhi Jin, and Hong Mei. 2024. Hot or cold? adaptive temperature sampling for code generation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 437–445.

Received 2025-09-12; accepted 2026-03-24